

Uso de consola o línea de comandos para la gestión de MongoDB

Introducción

Como muchas otras aplicaciones, MongoDB provee una herramienta CLI (Command Line Interface) para manipular los datos. Normalmente conocida como Mongo Shell, o *mongosh*, esta herramienta nos permite interactuar directamente con el motor de base de datos mediante comandos de una sintaxis casi idéntica al lenguaje JavaScript.

A través de esta interfaz, es posible crear bases de datos, colecciones y documentos, así como ejecutar operaciones de consulta, actualización y eliminación de datos.

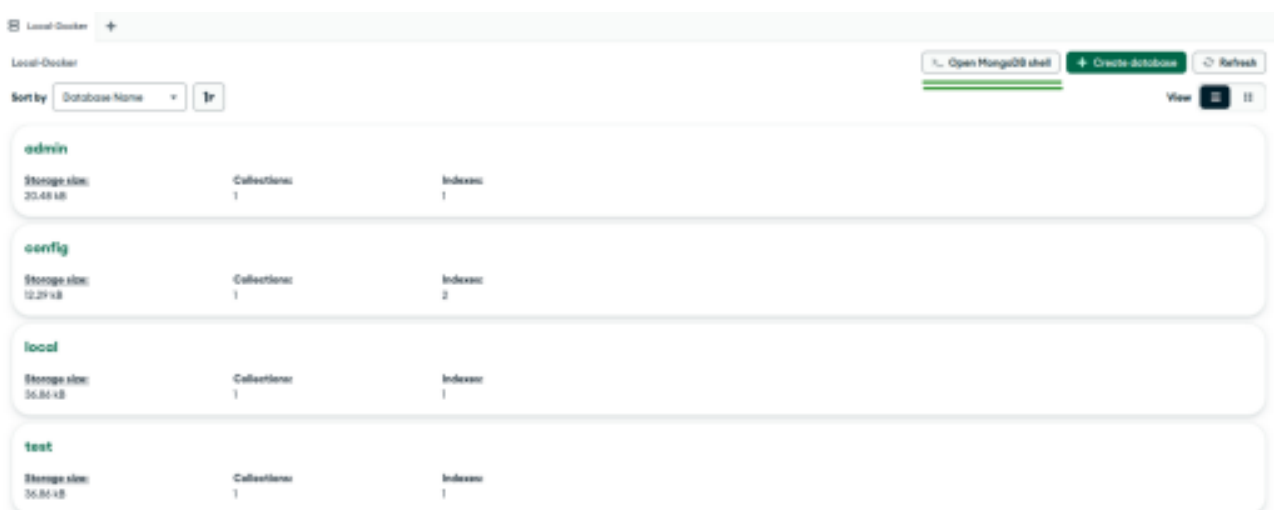
Comprender su funcionamiento es realmente útil para el modelado, la administración y la depuración de datos en entornos NoSQL.

Obviamente, no está demás recordar que existe la herramienta MongoDB Compass para realizar las mismas operaciones desde una interfaz gráfica, sin embargo, en algunos escenarios puede ser más eficiente el uso de la CLI.

En este caso vamos a repasar solo algunas de las operaciones que la herramienta nos permite usar, así que te invitamos a que descubras las muchas otras a tu ritmo y necesidad haciendo uso del sistema de sugerencia (IntelliSense) integrado en *mongosh*.

Preparación del entorno

Antes de comenzar a revisar este módulo, asegurate de tener acceso a una instancia de MongoDB (MongoDB Atlas o instancia local). Si realizaste la instalación completa local (MongoDB + MongoDB Compass), deberías tener acceso a *mongosh* escribiendo esa palabra en cualquier terminal que uses (PowerShell, CMD, Bash). En caso de no tenerla o estar trabajando con la instancia de MongoDB Atlas se puede acceder a *mongosh* a través de MongoDB Compass también:



Creación de mi primera Bases de datos

En MongoDB, las bases de datos se crean automáticamente cuando se inserta el primer documento, o cuando se realiza una operación con persistencia. Utilizamos el comando *use* para apuntar el entorno a una base de datos particular:

```
use universidad; // apunta operaciones hacia la base de datos "universidad"
```

Comandos básicos:

```
db; // Ver la base de datos actual
show dbs; // Mostrará un listado de las bases de datos ya creadas
show collections; // Listar todas las colecciones
exit; // Cerrar conexión
db.getName(); // Devolverá el nombre de la base de datos en uso.
db.dropDatabase(); // Eliminará la base de datos seleccionada
```

Importante: La base de datos NO aparecerá en *show dbs* hasta que tenga al menos una colección con datos. Para eso vamos a crear una colección:

Creación de colecciones

La colección se crea automáticamente al insertar el primer documento:

```
db.createCollection("estudiantes"); // Creará la colección "estudiantes"
```

De la misma manera podemos realizar una gran cantidad de operaciones sobre tal colección de acuerdo a lo que necesitemos, dentro de otras más comunes tenemos:

```
db.getCollectionNames() // Listará las colecciones de la base de datos
db.estudiantes.renameCollection("alumnos"); // Renombrará la colección "estudiantes"
db.alumnos.drop(); // Eliminará la colección "alumnos"
```

Inserción de Documentos

Podemos Insertar UN documento - **insertOne()**:

```
db.estudiantes.insertOne({  
  legajo: 12345,  
  nombre: "Juan Pérez",  
  edad: 21,  
  carrera: "Ingeniería en Sistemas",  
  email: "[email protected]",  
  activo: true  
})
```

O podemos insertar MULTIPLES documentos- **insertMany()**:

```
db.estudiantes.insertMany([  
  {  
    legajo: 12346,  
    nombre: "Ana Martínez" ,  
    edad: 23,  
    carrera: "Ingeniería Industrial" ,  
    email: "[email protected]" ,  
    activo: true  
  },  
  {  
    legajo: 12347,  
    nombre: "Carlos López" ,  
    edad: 20,  
    carrera: "Ingeniería en Sistemas" ,  
    email: "[email protected]" ,  
    activo: false  
  },  
  {  
    legajo: 12348,  
    nombre: "Laura Fernández" ,  
    edad: 22,  
    carrera: "Ingeniería Química" ,  
    email: "[email protected]" ,  
    activo: true  
  }  
])
```

```
db.estudiantes.find() // trae todos los documentos que contiene la colección "estudiantes"
```

```
db.estudiantes.findOne() // muestra un documento de estudiantes
```

```
db.estudiantes.findOne({ nombre: "Laura Fernandez" }) // busca por un dato específico
```

```
db.estudiantes.countDocuments() // devuelve la cantidad de documentos que contiene "estudiantes"
```

```
db.estudiantes.countDocuments({ carrera: "ingeniería en sistemas" }) // cuenta los documentos, esta vez filtra por carrera
```

Los documentos en MongoDB siguen el formato JSON con pares **clave-valor**:

```
{
  "_id": ObjectId( "507f1f77bcf86cd799439011" ), // ID único automático
  "nombre": "Juan Pérez" , // String
  "edad": 21 , // Number
  "activo": true , // Boolean
  "cursos": ["Base de Datos", "Algoritmos" ] , // Array
  "direccion": { // Objeto embebido
    "calle": "San Martín 123" ,
    "ciudad": "Rosario"
  },
  "fecha_ingreso": new Date("2023-03-15") // Date
}
```

Operadores relacionales, lógicos

Introducción

MongoDB proporciona un conjunto de operadores relacionales y lógicos que permiten realizar consultas avanzadas y manipulaciones precisas sobre los documentos almacenados. A diferencia de los sistemas relacionales tradicionales, las operaciones en MongoDB se expresan mediante objetos que describen los criterios de búsqueda o modificación. Estos operadores son esenciales para implementar operaciones **CRUD** (Create, Read, Update, Delete).

Operadores relacionales

Los operadores relacionales permiten establecer criterios y **condiciones de comparación entre campos y valores**. Se utilizan principalmente en las operaciones de lectura (find) o actualización (update) para filtrar documentos según criterios específicos. Algunos ejemplos para confeccionar estos filtros son:

Operador	Significado	Ejemplo
\$eq	Igual a	{ edad: { \$eq: 30 } }
\$ne	Distinto de	{ ciudad: { \$ne: "Madrid" } }
\$gt	Mayor que	{ edad: { \$gt: 18 } }
\$gte	Mayor o igual que	{ stock: { \$gte: 10 } }
\$lt	Menor que	{ precio: { \$lt: 100 } }
\$lte	Menor o igual que	{ puntuacion: { \$lte: 5 } }
\$in	Dentro de la lista	{ categoria: { \$in: ["Electrónica", "Ropa"] } }
\$nin	No está en la lista	{ categoria: { \$nin: ["Hogar", "Deporte"] } }

Si bien de esta manera afinamos la búsqueda, podemos ir más allá y combinar las estructuras que necesitemos utilizando los operadores lógicos, que veremos a continuación.

Operadores lógicos

Los operadores lógicos permiten combinar múltiples **condiciones de búsqueda** o filtrado dentro de una misma consulta, posibilitando las manipulaciones y consultas altamente precisas sobre las colecciones. Por ejemplo:

Operador	Significado	Ejemplo
\$and	Todas las condiciones	<pre>{ \$and: [{ edad: { \$gte: 18 } }, { ciudad: "Buenos Aires" }] }</pre>
\$or	Al menos una condición	<pre>{ \$or: [{ pais: "Argentina" }, { pais: "Chile" }] }</pre>
\$not	No se debe cumplir ninguna	<pre>{ edad: { \$not: { \$gt: 30 } } }</pre>
\$nor	Ninguna debe cumplirse	<pre>{ \$nor: [{ activo: true }, { rol: "admin" }] }</pre>

Estos operadores nos proporcionan una forma de realizar consultas para confeccionar reportes, filtros dinámicos y sistemas de búsqueda avanzada.